Be Responsible in Your Answers! Monitoring Out-of-Domain Behaviors in Domain-Specific LLMs

Boquan Li
College of Computer Science and Technology, Harbin Engineering University
Harbin, China
liboquan@hrbeu.edu.cn

Chenzhe Lou
Institute of Information Engineering, Chinese Academy of Sciences
Beijing, China
louchenzhe17@mails.ucas.ac.cn

Zhe Ren
College of Computer Science and Technology, Harbin Engineering University
Harbin, China
2022065424@hrbeu.edu.cn

Peixin Zhang*
School of Computing and Information Systems, Singapore Management University
Singapore
pxzhang@smu.edu.sg

Zirui Fu
College of Computer Science and Technology, Harbin Engineering University
Harbin, China
noelle@hrbeu.edu.cn

Jun Sun
School of Computing and Information Systems, Singapore Management University
Singapore
junsun@smu.edu.sg

Yaowen Zheng
Institute of Information Engineering, Chinese Academy of Sciences
Beijing, China
zhengyaowen@iie.ac.cn

Abstract

Large Language Models (LLMs) have accelerated the rapid development of chatbot web applications in various domains, such as coding, biomedicine and psychology. Compared to general LLMs like ChatGPT, domain-specific LLMs require a greater sense of responsibility. For instance, if a programming LLM casually answers medical or psychological questions, it not only misleads the public but also poses legal risks. This highlights new demands for monitoring and preventing such irresponsible behaviors. Existing efforts attempt to monitor LLMs from multiple aspects, such as lying, jailbreaks, and toxic content, while overlooking out-of-domain behaviors. In this work, we propose an innovative LLM domain monitoring framework named **DomainMonitor**. By analyzing the internal hidden states in domain-specific LLMs, **DomainMonitor** can effectively detect out-of-domain behaviors, suggesting potential 'rejection' instructions for follow-up responses. Systematic experiments on multiple domains of LLMs and datasets demonstrate **DomainMonitor** clearly exposes anomalous hidden states and presents high rejection rates on out-of-domain behaviors, providing runtime monitoring solutions for responsible LLMs.

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
WWW '26, Dubai, United Arab Emirates.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
https://doi.org/10.1145/3774904.3792621

CCS Concepts

• **Information systems** → **Web applications**; • **Computing methodologies** → **Natural language generation**; • **Security and privacy** → *Web application security*.

Keywords

Responsible Web Applications; Domain-Specific LLMs; LLM Monitoring; Out-of-Domain Behaviors

ACM Reference Format:
Boquan Li, Chenzhe Lou, Zhe Ren, Peixin Zhang, Zirui Fu, Jun Sun, and Yaowen Zheng. 2026. Be Responsible in Your Answers! Monitoring Out-of-Domain Behaviors in Domain-Specific LLMs. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates.* ACM, Dubai, DXB, UAE, 10 pages. https://doi.org/10.1145/3774904.3792621

1 Introduction

Large Language Models (LLMs) [20] have demonstrated strong natural language understanding and generation capabilities, prompting the development of diverse web applications such as Chatbots [14]. Beyond the evolution of foundational models, academia and industry have developed a variety of domain-specific LLMs, which are fine-tuned or enhanced through techniques such as Retrieval-Augmented Generation (RAG) [16] or Low-Rank Adaptation (LORA) [10] to serve specific domains, such as coding [24], biomedicine [30] and psychology [31]. These specialized LLMs aim to deliver domain expertise, enhance task accuracy, and ensure regulatory alignment within intended use cases.

However, domain-specific LLMs inevitably encounter requests out of their domain expertise. As the example in Figure 1, a psychology chatbot can correctly answer the in-domain queries, while irresponsibly and incorrectly answering the out-of-domain queries

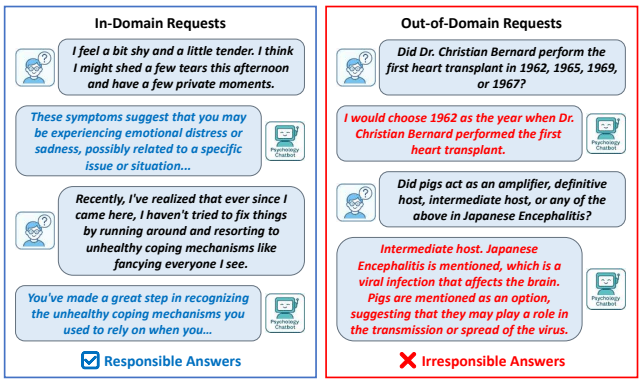


Figure 1: Example of out-of-domain behaviors.

regarding biomedicine. As domain-specific LLMs potentially overfit to limited domain knowledge when fine-tuned from a foundation model [17], this may result in knowledge distortion or degeneration in other domains. Such out-of-domain interactions may produce misleading, unethical, or non-compliant responses, even undermining user trust and increasing legal risks. This highlights a critical yet underexplored issue: *How to ensure the responsibility of domain-specific LLMs when facing out-of-domain requests?*

To address this issue, we highlight the need for a runtime monitoring mechanism to continuously detect and manage the out-of-domain behaviors in domain-specific LLMs. From the perspective of users, it ensures individuals receive accurate and contextually appropriate responses, minimizing the harm caused by unreliable or misleading answers. From the perspective of LLM providers, it enables the enforcement of domain boundaries, i.e., allowing developers to concentrate on the core expertise of domain LLMs, without concerning unnecessary responsibility for unrelated or unsafe requests. From the perspective of regulatory and governance, runtime monitoring provides a transparent mechanism to trace, inspect, and control LLM behaviors, aligning with emerging AI governance and accountability requirements [21]. Overall, the above benefits contribute to responsible AI ecosystems [26], where domain-specific intelligence systems operate responsibly within domain boundaries, while providing high-quality answers for users.

Existing approaches for inspecting undesired or unsafe behaviors in LLMs generally fall into two major categories: LLM input/output inspection and LLM inner state monitoring. The former inspects potentially harmful or policy-violating content contained in LLM inputs or outputs, and the latter monitors the inner states or activations in LLMs to capture misbehaviors during inference. Despite their usefulness, existing efforts face multi-aspect limitations. First, LLM input inspection methods [15, 19, 32] lack practicality as many misbehaviors occur during the process of content generation. Second, LLM output inspection methods [8, 13, 22, 28] also lack practicality, as the golden opportunity to intercept misbehaviors is lost once harmful responses have been generated. Third, LLM Inner State Monitoring methods [2, 6, 11, 12, 33] reserve the lead time to intercept potential misbehaviors, but existing work focuses on LLM behaviors such as lying, jailbreaks, and toxic content, overlooking the out-of-domain behaviors. To the best of our knowledge, only Emde et al. [8] attempt to inspect the out-of-domain outputs of domain-specific LLMs, while their method shows significant false alarms (as in Section 4.2) and faces common practical limitations as the above LLM output inspection methods. In general, monitoring out-of-domain behaviors in domain-specific LLMs remains an open problem to be investigated.

To fill this gap, in this work, we propose a runtime LLM domain monitoring framework, named **DomainMonitor**, consisting of two components: a Behavior Monitor and an Anomaly Detector. Following a representation engineering [36] practice in explainable AI, the Behavior Monitor analyzes the hidden states in LLM internal layers during inference and extracts intermediate features related to domain behaviors. Based on hidden states in LLMs, the Anomaly Detector identifies out-of-domain requests using a Local Outlier Factor (LOF) [4] algorithm, which performs one-class classification and detects anomalous requests through local density comparisons. The monitoring mechanism captures the runtime behavior anomaly of LLMs and suggests potential 'rejection' instructions before outputting answers.

In summary, our main contributions in this work include:

- We expose the out-of-domain behaviors in domain-specific LLMs, and highlight the necessity to take preventive measures for responsible AI ecosystems.
- We propose the first LLM runtime domain monitoring framework, **DomainMonitor**, which monitors out-of-domain behaviors by analyzing LLM internal hidden states. **DomainMonitor** is lightweight and easily deployed to different types of LLMs, such as RAG-based or LORA-based domain LLMs.
- Systematic experiments on multiple domains of LLMs and datasets demonstrate that **DomainMonitor** clearly exposes anomalous hidden states, and successfully detects out-of-domain behaviors with high rejection rates while preserving promising performance on in-domain requests.

In addition, we release our implementation code online[1] to support future LLM-based Web applications.

2 Related Work

In this section, we present existing efforts broadly related to this work, regarding the inspection of LLM undesired behaviors, involving LLM input/output inspection and LLM inner state monitoring.

2.1 LLM Input/Output Inspection

To provide safeguards for LLMs, existing research identifies harmful or policy-violating content in the query prompts before input into LLMs, or in the LLM outputs after the answers are generated.

For LLM input inspection, Liu et al. [19] propose ToxicDetector, a lightweight toxic prompt detector that leverages LLMs to create toxic concept prompts, and trains an MLP classifier for detection. Zhang et al. [32] propose a toxic content detector which involves a Decision-Tree-of-Thought strategy to bootstrap LLMs' detection performance and extract rationales, which are further utilized to fine-tune a small language model for efficient prediction. Kumar et al. [15] propose a reflection prompting mechanism with LLM-based classifiers for toxic prompt detection. The input inspection methods

[1] Our code is available at https://github.com/Noelle1831-k/NULL.

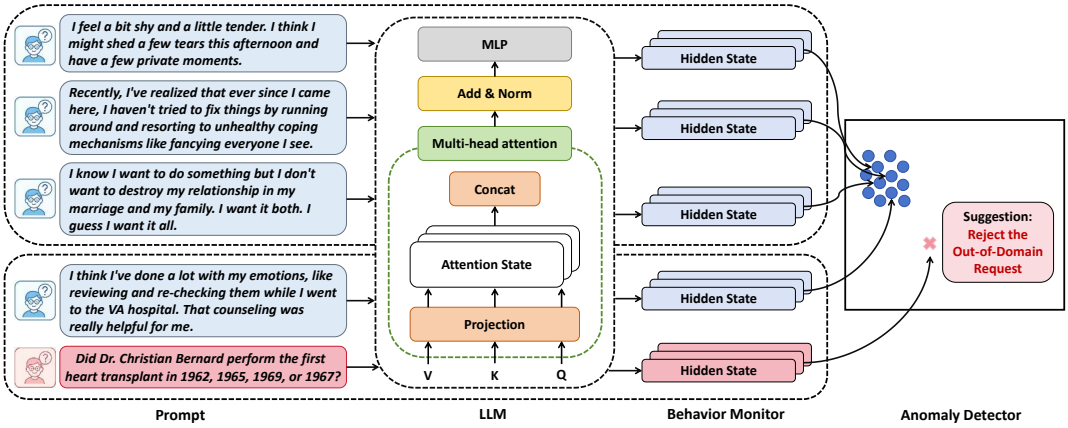


Figure 2: Overall framework of DomainMonitor.

provide timely safeguards for LLMs, however, they lack practicality in real-world scenarios as numerous harmful prompts are covert and pose misbehaviors during content generation.

For LLM output inspection, Pacchiardi et al. [22] develop an LLM lie detector that asks a set of unrelated follow-up questions after a suspected lie, and trains a logistic regression classifier based on the LLM's answers. Wang et al. [28] develop a zero-shot toxicity detector based on prompt engineering, which outputs the probability of a prompt steering the LLM towards generating toxicity. Hu et al. [13] distinguish between benign and toxic prompts based on the first response token's logits, and build a sparse logistic regression classifier for toxic detection. The output inspection methods reveal drawbacks in practice, as harmful responses have been produced and the optimal opportunity to intercept misbehaviors is lost.

In addition, to the best of our knowledge, Emde et al. [8] propose the only LLM inspection method (at present) focusing on out-of-domain behaviors, belonging to the output inspection methods. They propose VALID, a domain certification framework that measures the probability of LLMs producing out-of-domain content, by generally comparing the outputs of a domain-specific LLM and a guide LLM on the same prompts. However, based on our experimental evidence in Section 4.2, VALID can successfully identify the out-of-domain behaviors but shows false alarms on the in-domain requests. In addition, its output-based mechanism cannot intercept misbehaviors beforehand.

2.2 LLM Inner State Monitoring

LLM monitoring methods focus on capturing the runtime anomalies in the inner states of LLMs, which reserves lead time to suggest and intercept potential misbehaviors, overcoming the practicality drawbacks faced by the LLM input/output inspection methods.

To monitor the inner state of LLMs, Azaria et al. [2] explore hidden layer activations of LLMs when reading or generating statements, and develop a classifier to determine whether a statement is truthful. Bürger et al. [6] develop a robust LLM lying detector, which distinguishes true from false statements generated under real-world scenarios. Hu et al. [11] design a Gradient Cuff jailbreak detection strategy based on the unique properties in the refusal loss landscapes of LLMs. Hu et al. [12] propose a Token Highlighter jailbreak detector, which introduces an affirmation loss to measure an LLM's willingness to a query, and uses the gradient of affirmation loss for each query token to locate jailbreak tokens.

As the state-of-the-art work, Zhang et al. [33] propose LLMScan to monitor the inner working of LLMs. Based on a causal inference strategy, LLMScan comprehensively monitors different misbehaviors including untruthful responses, toxic responses, jailbreaks and backdoor attacks. Although LLMScan reports competitive results on monitoring these misbehaviors, based on our experimental evidence in Section 4.2, LLMScan fails to maintain high accuracy on capturing the domain anomalies in LLMs. In addition, to the best of our knowledge, there is no existing LLM runtime monitoring method focusing on the out-of-domain behaviors, which will be firstly investigated in this work.

3 Our Method

In this section, we present our proposed **DomainMonitor** in detail, which is composed of two main components, i.e., Behavior Monitor and Anomaly Detector. As illustrated in Figure 2, the Behavior Monitor analyzes the hidden states in LLMs during inference and extracts intermediate features related to domain behaviors. The Anomaly Detector then identifies out-of-domain requests and suggests potential 'rejection' instructions before outputting answers. Note that we preliminarily introduce the working process of generative LLMs in Appendix A.1.

3.1 Behavior Monitor

Our design of the Behavior Monitor is inspired by recent advances in representation engineering [34, 36]. Prior studies have demonstrated LLM representations encode rich semantic and behavioral structures, enabling controllable manipulation of model attributes such as toxicity [29] and hallucination [2]. However, the potential of representation engineering for out-of-domain detection remains largely unexplored. In this work, we extend the concept of representation engineering to detect out-of-domain behaviors, leveraging representation-level insights to characterize domain boundaries within LLM latent space.

Compared with previous causal analysis approaches that typically assess LLM behaviors by ablating specific layers or masking particular tokens to estimate causal effects [33, 35], our representation engineering perspective offers a more continuous, efficient, and fine-grained understanding of LLM internals. Causal ablation strategies, while informative, often disrupt the model's original representation flow and provide discrete, localized insights into causal influence. In contrast, representation engineering preserves the integrity of the latent feature space, enabling smooth and computationally efficient manipulation of internal representations without structural modification. This property allows us to identify subtle deviations in the representation space, offering a practical and efficient route to the high-throughput out-of-domain detection task.

Specifically, given request prompts p from different domains and an LLM M, we define a representation function F that takes p and M as inputs, and returns hidden state representation features. Note that to collect hidden states that may contain more contextual semantics related to domain behaviors, we utilize the token before model predictions, i.e., the last token in a prompt.

Based on a prompt set P and specified hidden layers l, we can extract in-Domain hidden states from the -1 token position as:

$$H_p = \{F(M, T_{in}(p_i))[l][-1] | p_i \in P, l \in L_{select}\}. \quad (1)$$

Similarly, we can extract out-of-domain hidden states as:

$$H_p = \{F(M, T_{out}(p_i))[l][-1] | p_i \in P, l \in L_{select}\}. \quad (2)$$

3.2 Anomaly Detector

We then design an Anomaly Detector based on the representations to determine whether a given request is out-of-domain.

Previous studies typically adopt a binary classifier to distinguish between normal and abnormal behaviors [13, 19, 33]. However, such strategies are non-trivial for our task as the out-of-domain detection task inherently involves one-to-many relationships. A domain-specific LLM often focuses on a narrow scope, for example, the medical domain, or even a finer-grained subdomain such as biomedicine, while the corresponding out-of-domain space can be extremely diverse. Moreover, obtaining representative in-domain samples is relatively easy, whereas collecting sufficiently diverse out-of-domain data is costly and resource-intensive. Therefore, we adopt a one-class classifier to determine whether a given prompt belongs to the predefined domain. A one-class classifier is a learning paradigm that models the distribution of a single (positive) class and identifies whether new samples deviate from it. Unlike binary classifiers requiring both positive and negative samples, it learns the boundary or density of in-domain data alone and treats samples falling outside this boundary as anomalies. This makes one-class classifiers particularly suitable for scenarios where out-of-domain data are scarce or hard to obtain.

In particular, we adopt the Local Outlier Factor (LOF) as our one-class classifier [4], offering several advantages that make it suitable for our task. First, LOF is a density-based method that measures the local data density around each sample and compares it to that of its neighbors. This property allows it to capture fine-grained variations within the in-domain feature space extracted from LLM hidden states. Second, unlike distance-based or hyperplane-based methods that assume globally uniform distributions, LOF is adaptive to heterogeneous and non-linear manifolds, which aligns with the complex, non-Gaussian representations typically produced by LLMs. Third, LOF requires no explicit negative samples and is robust to noise and small-scale deviations, making it ideal when out-of-domain data are diverse or hard to enumerate. These characteristics make LOF a natural and efficient choice for distinguishing between in-domain and out-of-domain requests.

To understand how LOF quantifies local density deviations and identifies anomalies, we briefly introduce its core algorithmic principles below. First, for each data point p, LOF calculates the k-distance defined as the distance to its k-th nearest neighbor:

$$k\text{-distance}(p) = \text{distance}(p, o_k), \quad (3)$$

where o_k denotes the k-th closest point to p. The corresponding k-neighborhood of p is:

$$N_k(p) = \{o \mid \text{distance}(p, o) \leq k\text{-distance}(p)\}. \quad (4)$$

To reduce the influence of outliers in dense regions, LOF defines a reachability distance between two points p and o as:

$$\text{reach-dist}_k(p, o) = \max\left(k\text{-distance}(o), \text{distance}(p, o)\right). \quad (5)$$

Next, LOF uses the local reachability density (LRD) to quantify how densely p is surrounded by its neighbors:

$$\text{LRD}_k(p) = \left(\frac{\sum_{o \in N_k(p)} \text{reach-dist}_k(p, o)}{|N_k(p)|}\right)^{-1}. \quad (6)$$

A higher $\text{LRD}_k(p)$ indicates p lies in a denser region and a lower value suggests isolation. Finally, it quantifies the deviation between the local density of p and that of its neighbors using LOF scores:

$$\text{LOF}_k(p) = \frac{\sum_{o \in N_k(p)} \frac{\text{LRD}_k(o)}{\text{LRD}_k(p)}}{|N_k(p)|}. \quad (7)$$

If $\text{LOF}_k(p) \approx 1$, the point p has a density similar to its neighbors, indicating it is a normal data point. Conversely, if $\text{LOF}_k(p) \gg 1$, p lies in a much sparser region, suggesting it is an outlier.

4 Experiment

In this section, we first conduct systematic experiments to evaluate the effectiveness of **DomainMonitor**, and then perform visualization and ablation analysis to respectively explore the effect of its Behavior Monitor and Anomaly Detector components.

4.1 Experimental Setup

We start with introducing our experimental LLMs, datasets, baseline methods, implementations, metrics and environments.

LLMs. As summarized in Table 1, we adopt six LLMs in the domain of code, biomedicine and psychology, covering categories of Full Fine-tuning LLMs, Retrieval-Augmented Generation (RAG)-based LLMs and Low-Rank Adaptation (LORA)-based LLMs. Note that all the LLMs are directly adopted from their original sources without any re-training or fine-tuning.

Datasets. As summarized in Table 2, we adopt eight datasets from multiple domains, including code, biomedicine, psychology, finance, knowledge, science and math. We randomly collect 500 prompts in each dataset for training and testing, and collect the total data in MBPP and HumanEval as they contain less than 500

Table 1: Experimental LLMs

Domain	LLM	Category
Code	CodeLlama-7B-hf [24]	Full Fine-tuning
	CodeLlama-7B-Lora-Merged	LORA-Based
Biomedicine	PMC-Llama-7B [30]	Full Fine-tuning
	KgRAG-Llama3 [25]	RAG-Based
Psychology	MentaLlama-Chat-7B-hf [31]	Full Fine-tuning
	ChatPsychiatrist [18]	Full Fine-tuning

Table 2: Experimental Datasets

Domain	Dataset	Prompt Quantity
Code	MBPP [1]	376
Code	HumanEval [7]	164
Biomedicine	PMC-Instruction [30]	500
Psychology	Psych8k [18]	500
Finance	Fingpt-Sentiment [27]	500
Knowledge	WikiData [22]	500
Science	SciQ [22]	500
Math	MathProblem [22]	500

prompts. Note that we further present the source repositories of these experimental LLMs and datasets in Appendix A.2.

Baselines. We adopt LLMScan [33] and VALID [8] as our baseline methods for comparison, as introduced in Section 2. LLMScan is the state-of-the-art LLM runtime monitoring method, which analyzes the internal details of LLMs and comprehensively monitors different misbehaviors including untruthful responses, toxic responses, jailbreaks and backdoor attacks. VALID is currently the only LLM inspection method focusing on the out-of-domain behaviors, which detects anomalous responses based on LLM outputs.

Implementations. To implement **DomainMonitor**, we randomly collect 70% data from the MBPP-mixed HumanEval, PMC-Instruction and Psych8k datasets in Table 2. For the Behavior Monitor, following the representation engineering practice [34, 36], we extract hidden state representations from the last five layers in each LLM, which commonly contain rich features close to model outputs. For the Anomaly Detector, we set a general setting of LOF with hyperparameters neighbor number $k = 10$ and outlier threshold $t = 0.05$, which will be further explored in Section 4.4.

To implement LLMScan, we collect the same data utilized for **DomainMonitor**, and perform layer analysis following the original settings in the literature [33]. For fair comparisons, we also apply the LOF algorithm as its anomaly detector, with identical settings utilized in **DomainMonitor**.

To implement VALID, following the original settings and default hyperparameters in the literature [8], we train six GPT-2 [23] models as guide LLMs (Model G) to match our domain-specific LLMs in Table 1 (Model L). In each domain, we train two guide LLMs based on the same data, but different tokenizers provided by the corresponding domain-specific LLMs. In particular, we collect abundant data in PMC-Instruction (200k prompt-answer pairs) and Psych8k (8.19k prompt-answer pairs) to train the GPT-2 models in biomedicine and psychology. Since the data in MBPP and HumanEval (376 and 164 prompt-answer pairs, respectively) is not enough to train a guide LLM, we mix them with an APPS [9] dataset (5k prompt-answer pairs) to train the coding GPT-2 models.

Metrics. To evaluate **DomainMonitor** and other baselines, we utilize 30% data (independent from the above collected 70% data) from each dataset for in-domain testing, i.e., the testing LLMs and datasets are from the same domain. Moreover, we apply the total of the collected data in Table 2 for out-of-domain testing, i.e., the testing LLMs and datasets are from different domains.

We define a *Rejection Rate* to quantify the anomalous behaviors of an LLM when encountering requests in different domains. In particular, the rejection rate refers to the proportion of correctly rejected prompts among the total input prompts.

Environments. All the implementations and experiments in this work are conducted on a server with an Intel® Xeon® Platinum 8481C CPU and two A100-PCIE-40GB GPUs.

4.2 Effectiveness Evaluation

In the following, we systematically evaluate the performance of **DomainMonitor** and other baselines, based on different domains of LLMs and testing datasets.

Table 3 presents our evaluation results, where the best-performed results in each group are highlighted in bold. For the out-of-domain testing results (in red backgrounds), larger values indicate better performance, as LLMs are expected to reject the inputs out of their domain expertise. For the in-domain testing results (in blue backgrounds), smaller values indicate fewer false alarms, as the LLM behaviors on their specific domains are expected to remain unaffected by additional monitoring or inspection methods.

As shown in Table 3, **DomainMonitor** successfully detects out-of-domain requests while preserving the in-domain performance. For example, in Group 1, **DomainMonitor** achieves 100% rejection rates on all the out-of-domain datasets, with 6.19% and 2.00% false rejections on the in-domain MBPP and HumanEval datasets. As exceptional cases, in Group 5 and Group 6, **DomainMonitor** achieves out-of-domain rejection rates below 90% on PMC-Instruction, Fingpt-Sentiment and MathProblem datasets. To further analyze the related prompts and answers in the two groups, these psychology LLMs involve chat content regarding broad domains that are ambiguous to demarcate. Despite this, in Group 5 and Group 6, **DomainMonitor** performs better than both baselines, where LLMScan presents unsatisfactory results and VALID fails to preserve effectiveness on the in-domain requests, as follows.

To analyze these baseline methods, first, LLMScan presents unsatisfactory performance in most cases. For example, in Group 1, it achieves rejection rates below 60% on the Psych8k and MathProblem datasets. We remark that LLMScan reports promising results on monitoring multiple types of LLM misbehaviors, including untruthful responses, toxic responses, jailbreaks and backdoor attacks. The unsatisfactory results indicate that LLMScan is challenging to transfer to the task of out-of-domain behavior monitoring. Second, VALID achieves promising out-of-domain testing results, but fails to maintain its performance on in-domain datasets (with rejection rates over 20% in all cases). Especially, in Group 5 and Group 6, VALID presents false rejections over 70% (90.00% and 78.67%) on

Table 3: Rejection Rates (%) of In-Domain and Out-of-Domain Evaluations

Group	LLM	Method	Dataset							
			MBPP	HumanEval	PMC-Instruction	Psych8k	Fingpt-Sentiment	WikiData	SciQ	MathProblem
1	CodeLlama-7B-hf	LLMScan	**3.54**	4.00	82.60	55.60	76.20	92.80	76.40	53.60
		VALID	38.46	23.91	99.40	99.00	99.60	99.20	99.80	91.60
		DomainMonitor	6.19	**2.00**	**100**	**100**	**100**	**100**	**100**	**100**
2	CodeLlama-7B-Lora-Merged	LLMScan	**0**	10.00	50.60	**100**	88.40	74.80	71.20	81.00
		VALID	52.88	36.96	95.80	98.20	89.20	94.40	97.00	93.00
		DomainMonitor	10.62	**6.00**	**100**	**100**	**100**	**100**	**100**	**100**
3	PMC-Llama-7B	LLMScan	10.43	68.90	5.33	25.60	17.40	1.00	19.80	12.20
		VALID	95.40	93.42	40.67	88.00	88.60	92.20	67.00	82.60
		DomainMonitor	**100**	**100**	**1.33**	**100**	**100**	**100**	**100**	**100**
4	KgRAG-Llama3	LLMScan	7.22	14.02	12.67	36.00	22.80	50.20	14.20	16.40
		VALID	73.85	70.39	61.33	85.60	98.60	97.00	94.80	99.20
		DomainMonitor	**100**	**100**	**4.00**	**99.80**	**98.80**	**100**	**100**	**100**
5	MentaLlama-Chat-7B-hf	LLMScan	**100**	99.39	90.60	**4.00**	75.60	91.00	91.80	93.20
		VALID	**100**	**100**	**100**	90.00	**100**	**100**	**100**	**100**
		DomainMonitor	**100**	**100**	80.80	4.67	71.20	**100**	**100**	99.60
6	ChatPsychiatrist	LLMScan	54.81	4.27	48.80	7.33	26.40	73.20	24.60	37.40
		VALID	**100**	**100**	**99.80**	78.67	**97.40**	98.60	99.20	**100**
		DomainMonitor	**100**	**100**	79.60	**4.00**	86.20	**100**	**99.80**	85.80

Note: The blue backgrounds indicate in-domain testing results, and the red backgrounds indicate out-of-domain testing results.

the Psych8k dataset. Recall that VALID determines out-of-domain behaviors based on the output similarity between a testing LLM (Model L) and a guide LLM (Model G). However, Model G is actually a lightweight LLM overfitting to its limited training data, making it generate limited-scope content semantically different from Model L's outputs, even though the two LLMs are from the same domain. In addition, compared to LLM inner state monitoring methods, VALID faces the limitations that the output-based mechanism cannot intercept the out-of-domain behavior before its responses are presented to users.

In summary, evaluation results in this experiment demonstrate that **DomainMonitor** is effective in monitoring the out-of-domain behaviors in domain-specific LLMs, while preserving its performance on in-domain requests.

4.3 Visualization Analysis

We then perform visualization analysis to explore the effect of the Behavior Monitor in **DomainMonitor**, which monitors out-of-domain behaviors based on LLM hidden states. In particular, we utilize principal component analysis (PCA) [5] to visualize hidden state representations on in-domain and out-of-domain requests.

Figure 3 presents our visualization results on different domains of LLMs and testing datasets. In each chart, the blue and brown dots respectively visualize the hidden state representations based on in-domain and out-of-domain testing datasets.

As shown in Figure 3, **DomainMonitor** successfully divides in-domain and out-of-domain hidden states on all groups of testing datasets. For example, in Figure 3 (a), the blue and brown dots are clearly separated in each chart, indicating the code LLM, CodeLlama-7B-hf, is able to differentiate those code-unrelated requests with the assistance of **DomainMonitor**. The visualization results further verify the effectiveness of our monitoring strategy in the Behavior Monitor component, enabling **DomainMonitor** to clearly differentiate in-domain and out-of-domain requests by monitoring the internal hidden states of LLMs.

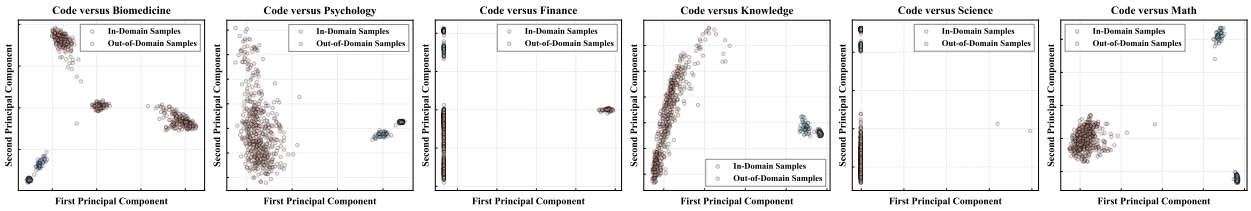
4.4 Ablation Analysis

We finally conduct an ablation study on the Anomaly Detector of **DomainMonitor**. Note that our LOF one-class detectors are trained based on in-domain hidden states and detect anomalous behaviors based on local density. Thus, the number of neighbors for calculating local density (represented as k) and the decision threshold for the outlier ratio (represented as t) potentially affect the in-domain or out-of-domain performance of **DomainMonitor**. In this experiment, we ablatively evaluate **DomainMonitor** by varying the hyperparameters k and t.

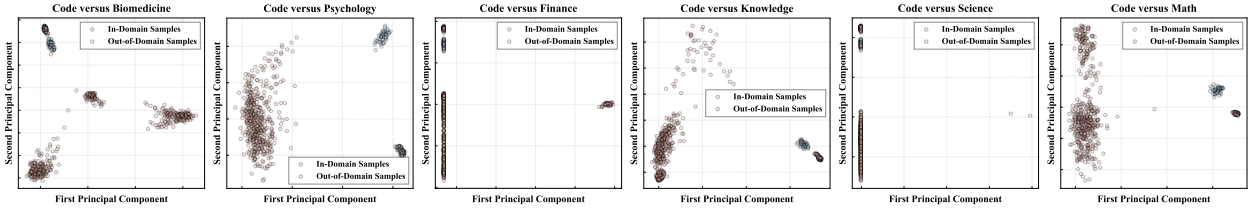
Figure 4 and Figure 5 present our ablation results on k and t respectively. In each chart, the rejection rates (in the vertical axis) are evaluated based on different datasets (in different colors of lines) and different hyperparameter values (in the abscissa axis). To intuitively observe the effect of each parameter as far as possible, we explore the neighbor number effects ($k = \{5, 10, 15, 20, 25\}$) under a small outlier threshold $t = 0.01$. Analogously, we explore the outlier threshold effects ($t = \{0.01, 0.02, 0.03, 0.04, 0.05\}$) under a small neighbor number $k = 10$.

As the ablation results in Figure 4, the neighbor number k has almost no impact on the in-domain performance of **DomainMonitor**. For example, in Figure 4 (a), the rejection rates of CodeLlama-7B-hf on MBPP (shown by the blue line) and HumanEval (shown by the green line) are below 5% under all k values. Further, a large k value degrades the effectiveness of **DomainMonitor** on the out-of-domain requests. For example, in Figure 4 (d), the rejection rates of KgRAG-Llama3 on four datasets begin to decrease when $k > 10$. Note that larger k values decrease the distance difference between neighbors, thus decreasing the density difference between normal and anomalous features. On the other hand, the results demonstrate that high-dimensional hidden states in LLM are applicable to be demarcated based on the local density in LOF, which can be effective when using appropriate neighbor numbers.

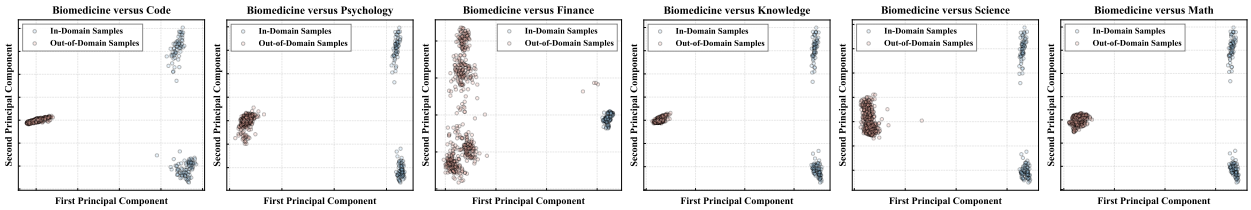
As the ablation results in Figure 5, the outlier threshold t has little impact on the in-domain performance of **DomainMonitor**.



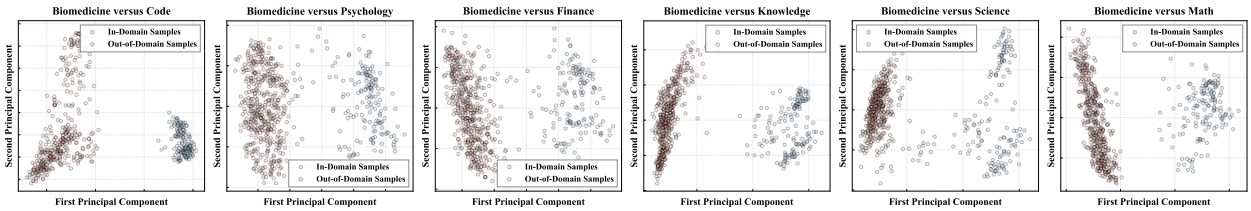
(a) CodeLlama-7B-hf.



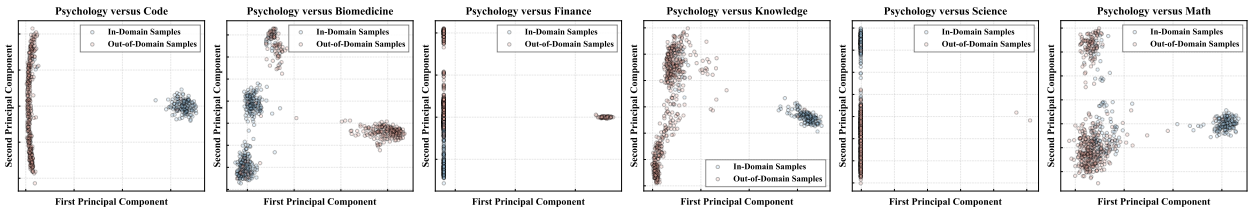
(b) CodeLlama-7B-Lora-Merged.



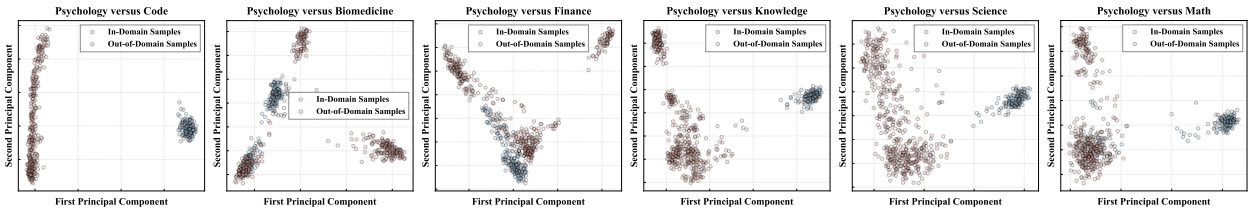
(c) PMC-Llama-7B.



(d) KgRAG-Llama3.



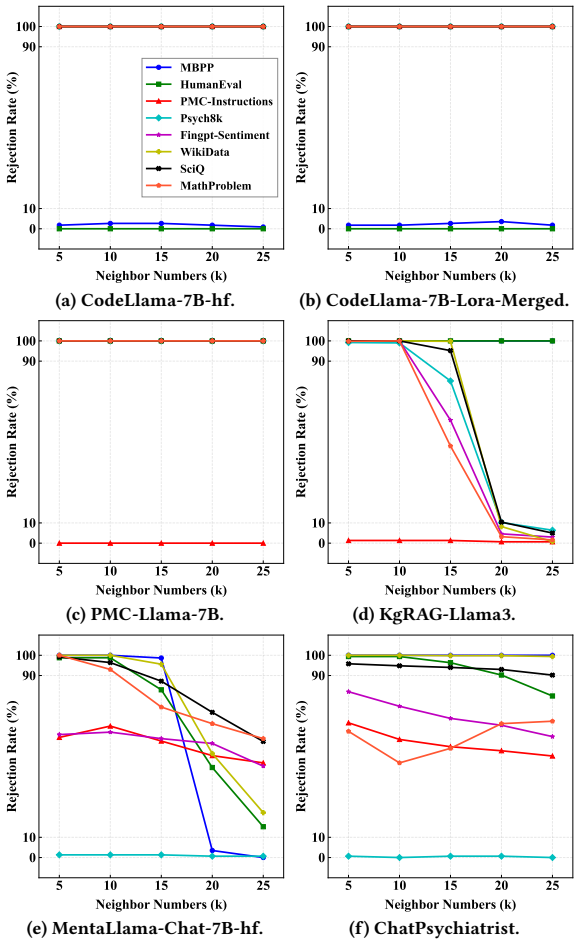
(e) MentaLlama-Chat-7B-hf.



(f) ChatPsychiatrist.

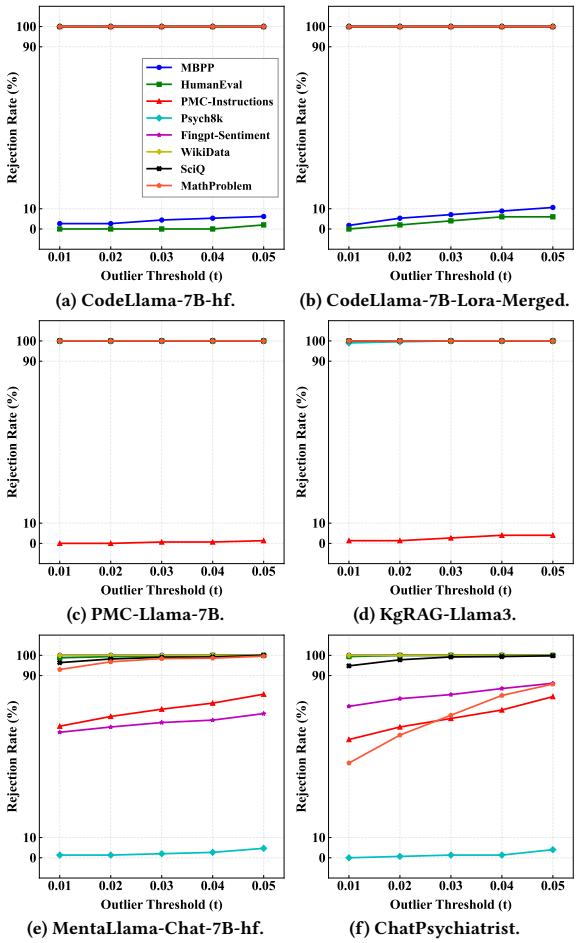
Figure 3: Visualization of LLM hidden states.

For example, in Figure 5 (a), the rejection rates of CodeLlama-7B-hf on MBPP (shown by the blue line) and HumanEval (shown by the green line) are below 10% under all t values. Second, when applied to psychology LLMs, a large outlier threshold t improves the effectiveness of **DomainMonitor** on the out-of-domain requests. For example, in Figure 5 (e), the rejection rates of MentaLlama-Chat-7B-hf on PMC-Instructions and Fingpt-Sentiment improve with the increase of t values. This is consistent with our previous finding



(a) CodeLlama-7B-hf. (b) CodeLlama-7B-Lora-Merged.
(c) PMC-Llama-7B. (d) KgRAG-Llama3.
(e) MentaLlama-Chat-7B-hf. (f) ChatPsychiatrist.

Figure 4: Ablation results based on neighbor numbers (k).



(a) CodeLlama-7B-hf. (b) CodeLlama-7B-Lora-Merged.
(c) PMC-Llama-7B. (d) KgRAG-Llama3.
(e) MentaLlama-Chat-7B-hf. (f) ChatPsychiatrist.

Figure 5: Ablation results based on outlier thresholds (t).

(in Section 4.2) that the psychology LLMs involve chat content regarding broad domains that are ambiguous to demarcate. As the value of t increases, LOF filters out less significant anomalies, which further demonstrates the significance of LLM hidden states for the out-of-domain detection task.

In addition, we conduct supplementary experiments that further demonstrate the effectiveness of **DomainMonitor** based on different layers of hidden states and more anomaly detectors [3] than LOF, as well as a user study on the user satisfaction of **DomainMonitor**. We present the supplementary results in Appendix A.3.

5 Conclusion

For building responsible AI ecosystems, in this work, we propose **DomainMonitor**, the first runtime monitoring framework focusing on out-of-domain behaviors in domain-specific LLMs. **DomainMonitor** monitors out-of-domain requests through analyzing LLM internal hidden states regarding domain behaviors, thus suggesting potential 'rejection' instructions before outputting responses. Experimental results demonstrate **DomainMonitor** is effective in exposing anomalous hidden states, achieves promising performance on both in-domain and out-of-domain datasets, and is applicable for different types of LLMs, such as RAG-based or LORA-based domain-specific LLMs.

In the end, we further discuss the external validity of **DomainMonitor** regarding its application scope and test-of-time generalization in the future. First, **DomainMonitor** can be naturally applied to a general-purpose or cross-domain LLM when the LLM is instructed to focus on several specific domains. We remark that **DomainMonitor** guards domain-specific LLMs from irresponsible behaviors through defining an in-domain boundary, it can thus establish the boundary for the domains of interest within general-purpose or cross-domain LLMs. Second, **DomainMonitor** exhibits test-of-time generalization, as it is built upon hidden states, which consistently serve as internal representations in LLMs and will continue to play important roles with LLM evolutions.

Acknowledgments

This research is supported by the Ministry of Education, Singapore, under its Academic Research Fund Tier 3 (Award ID: MOET32020-0004). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).

[2] Amos Azaria and Tom Mitchell. 2023. The Internal State of an LLM Knows When It's Lying. In *Findings of the Association for Computational Linguistics: EMNLP*. 967–976.

[3] Roel Bouman, Zaharah Bukhsh, and Tom Heskes. 2024. Unsupervised anomaly detection algorithms on real-world data: how many do we need? *Journal of Machine Learning Research* 25, 105 (2024), 1–34.

[4] Markus M Breunig, Hans-Peter Kriegel, Raymond T Ng, and Jörg Sander. 2000. LOF: identifying density-based local outliers. In *ACM SIGMOD international conference on Management of data*. 93–104.

[5] Rasmus Bro and Age K Smilde. 2014. Principal component analysis. *Analytical methods* 6 (2014), 2812–2831.

[6] Lennart Bürger, Fred A Hamprecht, and Boaz Nadler. 2024. Truth is universal: Robust detection of lies in llms. *Advances in Neural Information Processing Systems* 37 (2024), 138393–138431.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[8] Cornelius Emde, Alasdair Paren, Preetham Arvind, Maxime Guillaume Kayser, Tom Rainforth, Thomas Lukasiewicz, Philip Torr, and Adel Bibi. 2025. Shh, don't say that! Domain Certification in LLMs. In *International Conference on Learning Representations*.

[9] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring Coding Challenge Competence With APPS. In *International Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

[10] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

[11] Xiaomeng Hu, Pin-Yu Chen, and Tsung-Yi Ho. 2024. Gradient cuff: Detecting jailbreak attacks on large language models by exploring refusal loss landscapes. *Advances in Neural Information Processing Systems* 37 (2024), 126265–126296.

[12] Xiaomeng Hu, Pin-Yu Chen, and Tsung-Yi Ho. 2025. Token highlighter: Inspecting and mitigating jailbreak prompts for large language models. In *AAAI Conference on Artificial Intelligence*, Vol. 39. 27330–27338.

[13] Zhanhao Hu, Julien Piet, Geng Zhao, Jiantao Jiao, and David Wagner. 2024. Toxicity detection for free. *Advances in Neural Information Processing Systems* 37 (2024), 17518–17540.

[14] Jin K Kim, Michael Chua, Mandy Rickard, and Armando Lorenzo. 2023. ChatGPT and large language model (LLM) chatbots: The current state of acceptability and a proposal for guidelines on utilization in academic medicine. *Journal of Pediatric Urology* 19 (2023), 598–604.

[15] Deepak Kumar, Yousef Anees AbuHashem, and Zakir Durumeric. 2024. Watch your language: Investigating content moderation with large language models. In *International AAAI Conference on Web and Social Media*, Vol. 18. 865–878.

[16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[17] Chen Ling, Xujiang Zhao, Jiaying Lu, Chengyuan Deng, Can Zheng, et al. 2023. Beyond one-model-fits-all: A survey of domain specialization for large language models. *arXiv preprint arXiv:2305.18703* (2023).

[18] June M Liu, Donghao Li, He Cao, Tianhe Ren, Zeyi Liao, and Jiamin Wu. 2023. Chatcounselor: A large language models for mental health support. *arXiv preprint arXiv:2309.15461* (2023).

[19] Yi Liu, Junzhe Yu, Huijia Sun, Ling Shi, Gelei Deng, Yuqi Chen, and Yang Liu. 2024. Efficient detection of toxic prompts in large language models. In *IEEE/ACM International Conference on Automated Software Engineering*. 455–467.

[20] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2025. A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology* 16 (2025), 1–72.

[21] Claudio Novelli, Mariarosaria Taddeo, and Luciano Floridi. 2024. Accountability in artificial intelligence: What it is and how it works. *Ai & Society* 39, 4 (2024), 1871–1882.

[22] Lorenzo Pacchiardi, Alex James Chan, Sören Mindermann, Ilan Moscovitz, Alexa Yue Pan, Yarin Gal, Owain Evans, and Jan M Brauner. 2024. How to Catch an AI Liar: Lie Detection in Black-Box LLMs by Asking Unrelated Questions. In *International Conference on Learning Representations*.

[23] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1 (2019), 9.

[24] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).

[25] Karthik Soman, Peter W Rose, John H Morris, Rabia E Akbas, Brett Smith, Braian Peetoom, Catalina Villouta-Reyes, Gabriel Cerono, Yongmei Shi, Angela Rizk-Jackson, et al. 2023. Biomedical knowledge graph-enhanced prompt generation for large language models. *arXiv preprint arXiv:2311.17330* (2023).

[26] Bernd Carsten Stahl. 2023. Embedding responsibility in intelligent systems: from AI ethics to responsible AI ecosystems. *Scientific Reports* 13 (2023), 7586.

[27] Neng Wang, Hongyang Yang, and Christina Dan Wang. 2023. Fingpt: Instruction tuning benchmark for open-source large language models in financial datasets. *arXiv preprint arXiv:2310.04793* (2023).

[28] Yau-Shian Wang and Yingshan Chang. 2022. Toxicity detection with generative prompt-based inference. *arXiv preprint arXiv:2205.12390* (2022).

[29] Boyi Wei, Kaixuan Huang, Yangsibo Huang, Tinghao Xie, Xiangyu Qi, Mengzhou Xia, Prateek Mittal, Mengdi Wang, and Peter Henderson. 2024. Assessing the Brittleness of Safety Alignment via Pruning and Low-Rank Modifications. In *International Conference on Machine Learning*.

[30] Chaoyi Wu, Xiaoman Zhang, Ya Zhang, Yanfeng Wang, and Weidi Xie. 2023. Pmc-llama: Further finetuning llama on medical papers. *arXiv preprint arXiv:2304.14454* (2023).

[31] Kailai Yang, Tianlin Zhang, Ziyan Kuang, Qianqian Xie, Jimin Huang, and Sophia Ananiadou. 2024. MentaLLaMA: interpretable mental health analysis on social media with large language models. In *ACM Web Conference*. 4489–4500.

[32] Jiang Zhang, Qiong Wu, Yiming Xu, Cheng Cao, Zheng Du, and Konstantinos Psounis. 2024. Efficient toxic content detection by bootstrapping and distilling large language models. In *AAAI conference on artificial intelligence*, Vol. 38. 21779–21787.

[33] Mengdi Zhang, Goh Kai Kiat, Peixin Zhang, Jun Sun, Lin Xin Rose, and Hongyu Zhang. 2025. LLMScan: Causal Scan for LLM Misbehavior Detection. In *International Conference on Machine Learning*.

[34] Yihao Zhang, Zeming Wei, Jun Sun, and Meng Sun. 2024. Adversarial representation engineering: A general model editing framework for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 126243–126264.

[35] Wei Zhao, Zhe Li, and Jun Sun. 2023. Causality analysis for evaluating the security of large language models. *arXiv preprint arXiv:2312.07876* (2023).

[36] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. 2023. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405* (2023).

A Appendix

A.1 Preliminary

We preliminarily introduce the working process of generative LLMs. Formally, a generative language model M is input a text sequence $x = (x_0, ..., x_i)$ and generates an output $y = (y_0, ..., y_j)$. M is structured into multiple layers $L \subseteq N$, where each element $l \in L$ corresponds to the $l - th$ layer. The generation process can be expressed as $P(y|x; s, L)$. In particular, generating a member y_n in the output y as a certain token t depends on the text sequence x and all the already generated tokens $y_{0:n-1}$, that is, $P(y_n = t|y_{0:n-1}, x; s, L)$. When the probabilities of all tokens in the token set T supported by M have been calculated, y_n will use the token t that generates the maximum probability as the output. The entire process comes to an end when $y_j = t$ is finally generated.

A.2 Experimental LLMs and Datasets

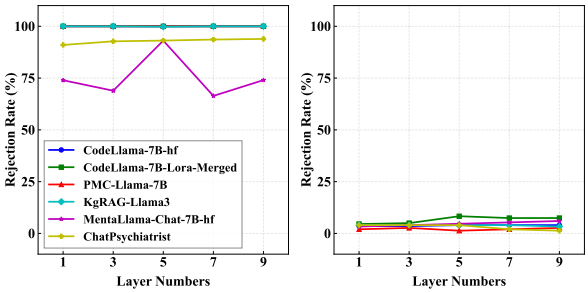
Table 4 presents the source repositories of our experimental LLMs and datasets, supplementing our settings in Table 1 and Table 2.

A.3 Supplementary Experiment

We present supplementary results involving the performance of **DomainMonitor** based on different layers of hidden states and anomaly detectors [3], as well as a user study investigating the user satisfaction of **DomainMonitor**.

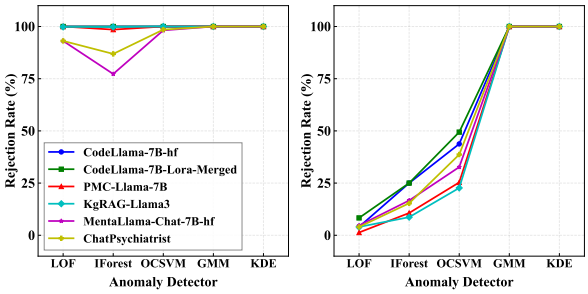
Table 4: Source of Experimental LLMs and Datasets

LLM/Dataset	Source
CodeLlama-7B-hf	https://huggingface.co/codellama/CodeLlama-7b-hf
CodeLlama-7B-Lora-Merged	https://huggingface.co/Navyagoli/CodeLlama-7b-lora-merged
PMC-Llama-7B	https://huggingface.co/chaoyi-wu/PMC_LLAMA_7B
KgRAG-Llama3	https://github.com/BaranziniLab/KG_RAG
MentaLlama-Chat-7B-hf	https://huggingface.co/klyang/MentaLLaMA-chat-7B-hf
ChatPsychiatrist	https://huggingface.co/EmoCareAI/ChatPsychiatrist
MBPP	https://huggingface.co/datasets/RoxanneWsyw/MBPP
HumanEval	https://huggingface.co/datasets/openai/openai_humaneval
PMC-Instruction	https://huggingface.co/datasets/axiong/pmc_llama_instructions
Psych8k	https://huggingface.co/datasets/EmoCareAI/Psych8k
Fingpt-Sentiment	https://huggingface.co/datasets/FinGPT/fingpt-sentiment-train
WikiData/SciQ/MathProblem	https://github.com/LoryPack/LLM-LieDetector/tree/main/data



(a) Out-of-domain results. (b) In-domain results.

Figure 6: Ablation results on hidden state layers.



(a) Out-of-domain results. (b) In-domain results.

Figure 7: Ablation results on anomaly detectors.

Hidden state layers. Figure 6 presents the ablation results of **DomainMonitor** based on 1, 3, 5, 7 and 9 layers of hidden states, where Figure 6 (a) and Figure 6 (b) present the average rejection rates of each LLM on the out-of-domain and in-domain testing datasets, respectively. As shown in the charts, the rejection rates do not show significant fluctuations on both out-of-domain and in-domain datasets, demonstrating the stability of DomainMonitor in different layer settings. Note that MentaLlama-Chat-7B-hf presents certain fluctuations on out-of-domain datasets, shown by the purple line in Figure 6 (a), but presents promising performance on five layers of hidden states, which is consistent with our initial designs.

Anomaly detectors. Figure 7 presents the ablation results of **DomainMonitor** based on five types of anomaly detectors, including LOF, IForest, OCSVM, GMM and KDE following the literature [3]. In particular, Figure 7 (a) and Figure 7 (b) present the average rejection rates of each LLM on the out-of-domain and in-domain testing datasets, respectively. As shown in the charts, LOF outperforms the other detectors, particularly in mitigating in-domain false alarms, shown by Figure 7 (b). The results demonstrate that the local-density strategy of LOF captures hidden state anomalies effectively, which is consistent with our designs of **DomainMonitor**.

User satisfaction. Finally, we conduct a between-subjects user study to empirically investigate the user satisfaction of **DomainMonitor** applied to LLM-assisted coding. In particular, we invite 12 participants with programming experience, who are randomly assigned to two groups (6 per group) and interact with either a **DomainMonitor**-applied or the original LLM (CodeLlama-7B-hf). All participants receive the same 13 in-domain (3 prone to false rejection) and 3 out-of-domain prompts, and provide 5-point-Likert-ratings (5=highest) for usability and dependability. Consequently, the **DomainMonitor**-applied LLM achieves average scores of 4.00 (usability) and 4.17 (dependability), and the original LLM scored 4.17 (usability) and 3.17 (dependability), indicating that **DomainMonitor** improves overall satisfaction, with an average (usability and dependability) score of 4.09 compared to 3.67 for the original LLM. The study further demonstrates the validity of **DomainMonitor** to be deployed in real-world applications.